# House Price Prediction

Data Science & Machine Learning Project

Final Project Report


Badr University in Assiut

Faculty of Computers and Information

| Team Member | Role |
|---|---|
| Saad Fakry | Data Analysis & Machine Learning |
| Riyad Mohammed | Data Analysis & Machine Learning |
| Samer Omar | Data Analysis & Machine Learning |
| Abdalah Said | Report Writing & Visualization |
| Shady Milad | Report Writing & Visualization |



Academic Year 2025 – 2026 | May 2026

# Table of Contents

# 1 Introduction

Real estate pricing is a complex and multifaceted problem influenced by dozens of structural, locational, and temporal factors. This project applies machine learning and data science techniques to predict house sale prices using the well-known **Ames Housing dataset**, originally compiled by Dean De Cock and containing 81 features describing residential properties sold in Ames, Iowa.

The primary objective is to build and compare regression models — **Linear Regression**, **Ridge Regression**, and **Lasso Regression** — that accurately predict sale prices, and to discover hidden groupings in the data using **K-Means Clustering**. All stages from raw data ingestion through model evaluation are presented in this report.

## Project Goals

- Clean and preprocess the raw dataset (handle missing values, encode categories).
- Perform exploratory data analysis to understand price distribution and feature correlations.
- Train three regression models and evaluate them using MSE and RMSE.
- Apply K-Means clustering to discover natural groupings of houses.
- Interpret and communicate insights from the models in a professional report.

# 2 Dataset Description

The dataset used is the **Ames Housing Dataset (train.csv)** containing 1,460 rows and 81 columns. Each row represents a single residential property sale. Features include numerical measurements (areas, year built) and categorical descriptors (zoning, neighborhood, roof style). The target variable is **SalePrice** — the actual sale price of the property in US dollars.

## Key Dataset Statistics

| Metric | Value |
|---|---|
| Total Rows | 1,460 |
| Total Columns | 81 |
| Numeric Features | 38 |
| Categorical Features | 43 |
| Target Variable | SalePrice |
| Min Sale Price | $34,900 |
| Max Sale Price | $755,000 |
| Mean Sale Price | ~$180,921 |

## Selected Column Descriptions

| Column | Type | Description |
|---|---|---|
| Id | Integer | Unique identifier for each house |

| Column | Type | Description |
|---|---|---|
| MSSubClass | Integer | Building class code |
| MSZoning | Categorical | General zoning classification |
| LotFrontage | Float | Linear feet of street connected to property |
| LotArea | Integer | Lot size in square feet |
| OverallQual | Integer | Overall material and finish quality (1–10) |
| OverallCond | Integer | Overall condition rating (1–10) |
| YearBuilt | Integer | Original construction year |
| GrLivArea | Integer | Above grade living area in sq ft |
| TotalBsmtSF | Integer | Total basement area in sq ft |
| GarageArea | Integer | Garage size in sq ft |
| GarageCars | Integer | Garage capacity in car units |
| SalePrice | Integer | TARGET: Property sale price in USD |

# 3 Team Composition

This project was completed as a team assignment for the Data Science course. All members participated in data exploration, model building, and report preparation.

| Member | Responsibility |
|---|---|
| Saad Fakry | Data Cleaning, Missing Value Imputation |
| Riyad Mohammed | EDA, Visualizations (Heatmap, Distribution) |
| Samer Omar | Regression Models (Linear, Ridge, Lasso) |
| Abdalah Said | K-Means Clustering, Elbow Method |
| Shady Milad | Report Writing, Code Review, Presentation |

# 4 Data Preprocessing

## 4.1 Initial Exploration

Upon loading the dataset, three initial inspection steps were performed: `df.head()` to preview the first five rows, `df.info()` to examine data types and null counts, and `df.describe()` to obtain descriptive statistics.

## 4.2 Dropping High-Missingness Columns

Four columns were identified as having over 80% missing values and were dropped entirely, reducing the dataset from 81 to 77 columns.

| Column | Missing Values | % Missing | Action |
|---|---|---|---|
| PoolQC | 1,453 | 99.5% | Dropped |
| MiscFeature | 1,406 | 96.3% | Dropped |
| Alley | 1,369 | 93.8% | Dropped |
| Fence | 1,179 | 80.8% | Dropped |

## 4.3 Encoding Categorical Features

Categorical columns were converted to numerical format using One-Hot Encoding via `pandas.get_dummies()` with `drop_first=True` to avoid multicollinearity.

```
df_encoded = pd.get_dummies(df, drop_first=True)
```

## 5 Handling Missing Values

Remaining missing values were imputed using statistically appropriate strategies — mean for normally distributed numerical features, median for skewed numerical features, and mode for categorical features.

| Column | Strategy | Justification |
|---|---|---|
| LotFrontage | Mean | Numeric — roughly symmetric distribution |
| MasVnrArea | Mean | Numeric — relatively low skew |
| GarageYrBlt | Median | Numeric — skewed by older properties |
| MasVnrType | Mode | Categorical — most common brick type |
| FireplaceQu | Mode | Categorical — ordinal quality rating |
| GarageFinish | Mode | Categorical — finish type |
| GarageQual | Mode | Categorical — quality rating |
| GarageCond | Mode | Categorical — condition rating |
| GarageType | Mode | Categorical — garage attachment type |
| BsmtExposure | Mode | Categorical — basement exposure level |
| BsmtFinType1 | Mode | Categorical — basement finish type |
| BsmtFinType2 | Mode | Categorical — secondary finish type |
| BsmtCond | Mode | Categorical — basement condition |
| BsmtQual | Mode | Categorical — basement quality |
| Electrical | Mode | Categorical — electrical system type |

✔ *After imputation,* `df.isnull().sum()` *confirmed* ***ZERO missing values*** *across all 77 columns.*

# 6 Exploratory Data Analysis (EDA)

## 6.1 Sale Price Distribution

A histogram with KDE overlay was plotted for the SalePrice column. The distribution is right-skewed — most houses are priced between $100,000–$200,000 while a smaller number of luxury properties push the tail toward $755,000.

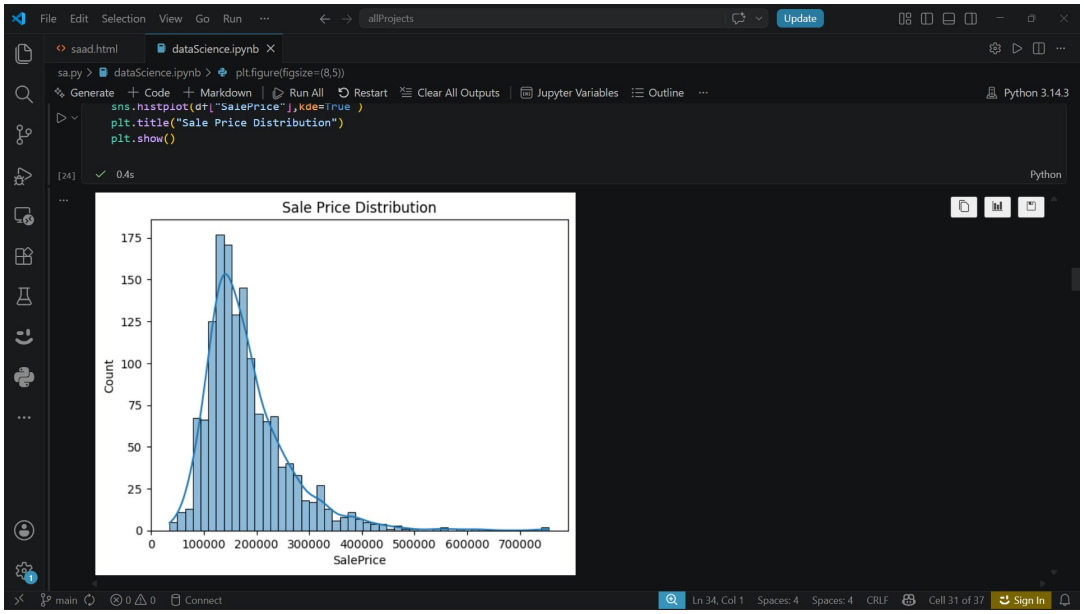


*Figure 1 — Sale Price Distribution (Histogram + KDE)*

***Key Insight:*** *The right-skewed distribution indicates that the majority of houses fall in the affordable-to-mid range. Outliers in the high price range could affect model performance if not handled carefully.*

## 6.2 Correlation Heatmap

A correlation heatmap was generated for all numerical features, revealing that several features have strong positive correlations with SalePrice.

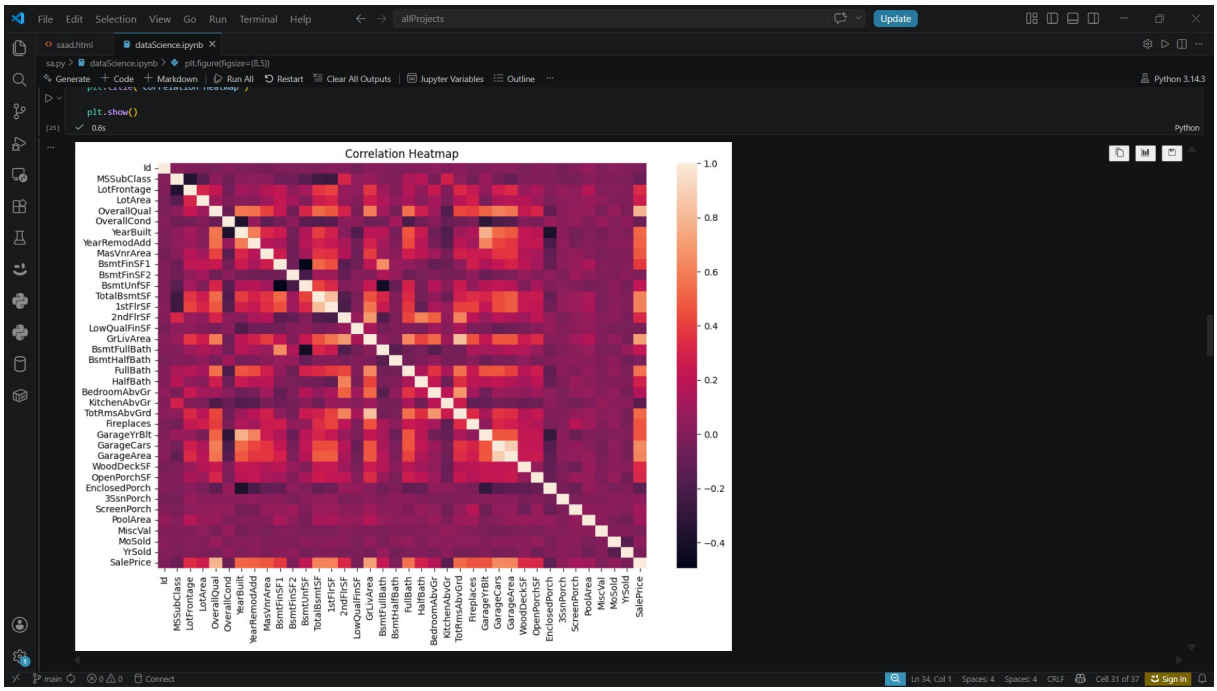


*Figure 2 — Correlation Heatmap of All Numerical Features*

| Feature | Correlation with SalePrice | Interpretation |
|---|---|---|
| OverallQual | ~0.79 | Higher quality = higher price (strongest) |
| GrLivArea | ~0.71 | Larger living area = higher price |
| GarageCars | ~0.64 | More garage space = higher price |
| GarageArea | ~0.62 | Garage area strongly linked to price |
| TotalBsmtSF | ~0.61 | Larger basement = higher price |
| YearBuilt | ~0.52 | Newer houses tend to sell for more |

# 7 Machine Learning Models

The dataset was split into training (80%) and testing (20%) sets using sklearn's `train_test_split` with `random_state=42` for reproducibility.

```
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

## 7.1 Linear Regression

Linear Regression is the baseline model. It fits a straight-line relationship between each feature and the target variable by minimizing the sum of squared residuals.

```
lr = LinearRegression() lr.fit(X_train, y_train) y_pred = lr.predict(X_test)
```

| Metric | Value |
|---|---|
| MSE | 2,736,422,343 |
| RMSE | $52,311 |

The RMSE of ~$52,311 means predictions deviate from actual sale price by ~$52,000 on average. This suggests overfitting as the model tries to fit every feature equally without penalization.

## 7.2 Ridge Regression (L2 Regularization)

Ridge Regression adds an L2 penalty term to the cost function, penalizing large coefficients and reducing overfitting. A small `alpha=0.1` was used.

```
ridge = Ridge(alpha=0.1) ridge.fit(X_train, y_train) ridge_pred = ridge.predict(X_test)
```

| Metric | Value |
|---|---|
| MSE | 1,111,118,708 |
| RMSE | $33,333 |

***Ridge Regression achieved the BEST performance*** *with RMSE = $33,333 — a* ***36% improvement*** *over Linear Regression. The L2 penalty successfully reduced overfitting by shrinking large coefficients without eliminating any features.*

## 7.3 Lasso Regression (L1 Regularization)

Lasso Regression applies an L1 penalty. Unlike Ridge, Lasso can shrink some coefficients to exactly zero, performing automatic feature selection. `alpha=0.1` was used.

```
lasso = Lasso(alpha=0.1) lasso.fit(X_train, y_train) lasso_pred = lasso.predict(X_test)
```

| Metric | Value |
|---|---|
| MSE | 2,715,473,965 |
| RMSE | $52,110 |

Lasso performed similarly to Linear Regression. With only alpha=0.1, the regularization effect is minimal. A higher alpha would allow Lasso to eliminate irrelevant features and potentially improve performance.

## 8 Model Comparison

| Model | MSE | RMSE | Rank |
|---|---|---|---|
| Linear Regression | 2,736,422,343 | $52,311 | **2nd** |
| Ridge Regression | 1,111,118,708 | $33,333 | **1st (Best)** |
| Lasso Regression | 2,715,473,965 | $52,110 | **3rd** |

Ridge Regression is clearly the best-performing model, reducing RMSE by approximately $19,000 compared to Linear Regression. The L2 regularization mechanism is well-suited to this dataset because many features are correlated (e.g., GarageArea and GarageCars), and Ridge handles multicollinearity more gracefully than standard Linear Regression.

# 9 K-Means Clustering

## 9.1 Elbow Method

To determine the optimal number of clusters K, the Elbow Method was applied. K-Means was run for K = 1 to 10, and the inertia was recorded for each value. The curve shows a clear elbow at **K=3**.

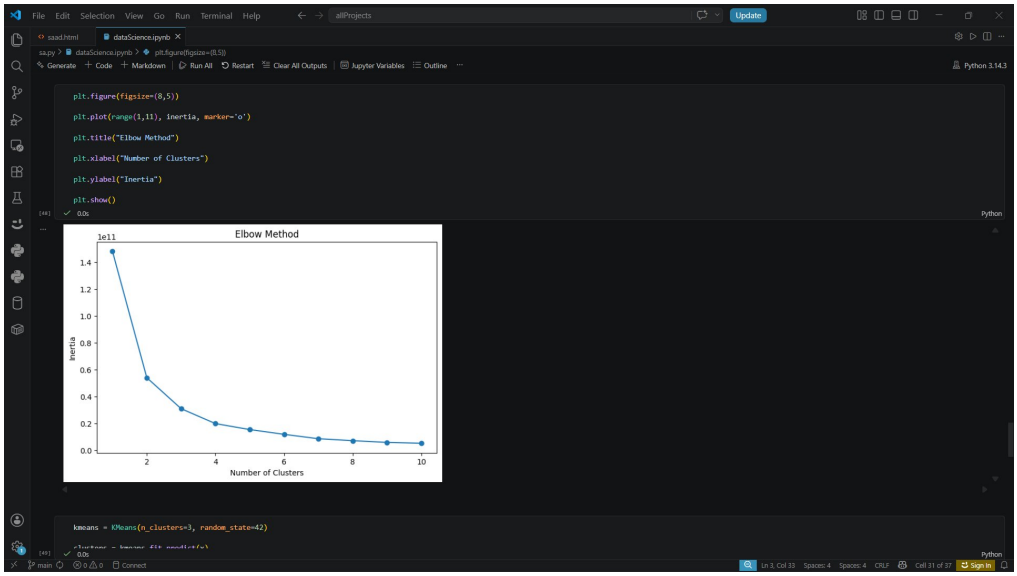


*Figure 3 — Elbow Method: Inertia vs. Number of Clusters*

## 9.2 Applying K-Means with K=3

K-Means was applied with `n_clusters=3` to segment the 1,460 houses into three distinct groups. A new column 'Cluster' was added to df_encoded.

```
kmeans = KMeans(n_clusters=3, random_state=42) clusters = kmeans.fit_predict(x)
df_encoded['Cluster'] = clusters
```

## 9.3 Cluster Analysis

| Cluster | Mean Sale Price | Interpretation |
|---|---|---|
| Cluster 0 | $178,171 | Economy / Entry-level Houses |
| Cluster 1 | $295,738 | Luxury / High-end Houses |
| Cluster 2 | $250,513 | Mid-range / Average Houses |

## 9.4 Boxplot Analysis

A boxplot of SalePrice by Cluster was generated using seaborn, confirming three distinct pricing tiers.

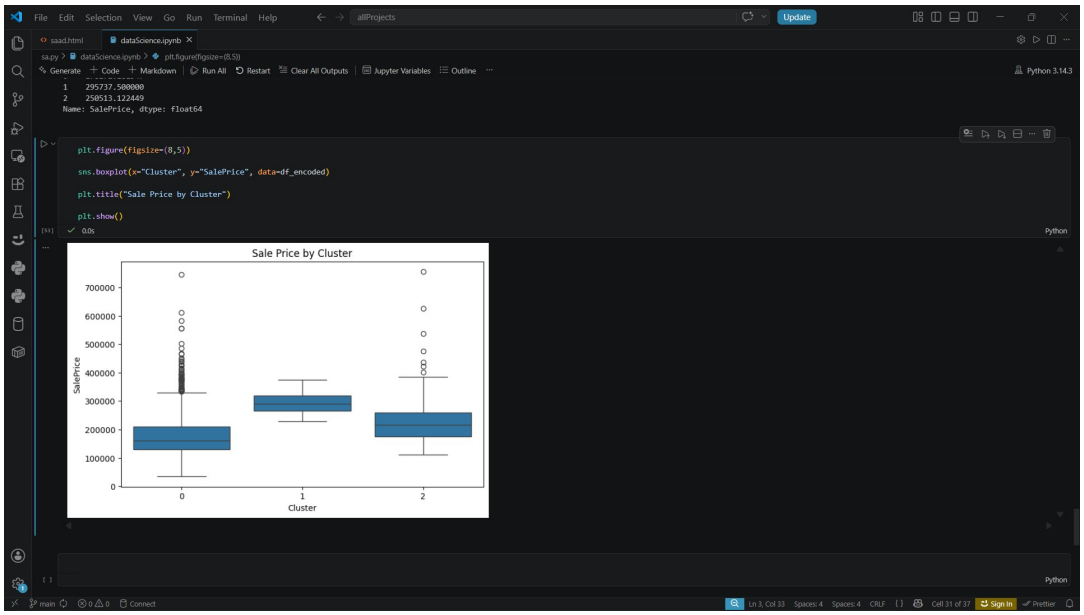


*Figure 4 — Sale Price Distribution by Cluster (Boxplot)*

- **Cluster 0 — Economy:** Lowest median price, wider spread, notable outliers.
- **Cluster 1 — Luxury:** Highest median price, compact distribution — luxury houses are consistently expensive.
- **Cluster 2 — Mid-range:** Median falls between Clusters 0 and 1, moderate spread.

***Clustering Insight:*** *K-Means successfully divided the housing market into three economically meaningful segments without using SalePrice as a direct input. This demonstrates that house features alone (size, quality, garage, etc.) naturally group into affordable, mid-range, and luxury tiers.*

# 10 Conclusion

This project successfully applied a complete data science pipeline — from raw data ingestion to machine learning model evaluation — on the Ames Housing dataset.

| Stage | Outcome |
|---|---|
| Data Cleaning | 4 columns dropped; all missing values imputed using mean/median/mode |
| EDA | Right-skewed price distribution; OverallQual is the strongest predictor |
| Linear Regression | RMSE = $52,311 — solid baseline with some overfitting |
| Ridge Regression | RMSE = $33,333 — best model; L2 regularization effective |
| Lasso Regression | RMSE = $52,110 — similar to linear; higher alpha recommended |
| K-Means | 3 clusters identified: economy, mid-range, and luxury housing |

The key takeaway is that **Ridge Regression outperforms** both Linear and Lasso models for this dataset, largely due to the high degree of multicollinearity among housing features. Unsupervised learning via K-Means produced meaningful and interpretable market segments that align with real-world understanding of the housing market.

## Future Work

Testing ensemble methods (Random Forest, Gradient Boosting) and applying log-transformation to SalePrice to address skewness could further improve prediction accuracy.

## 11 Appendix — Full Code

```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
from sklearn.cluster import KMeans

# Load & Explore Data
df = pd.read_csv('train.csv')
pd.set_option('display.max_columns', None)
df.head(); df.info(); df.describe()
df.isnull().sum().sort_values(ascending=False)

# Drop High-Missingness Columns
df = df.drop(['PoolQC','MiscFeature','Alley','Fence'], axis=1)

# Impute Missing Values
df['LotFrontage'] = df['LotFrontage'].fillna(df['LotFrontage'].mean())
df['GarageYrBlt'] = df['GarageYrBlt'].fillna(df['GarageYrBlt'].median())
df['MasVnrArea'] = df['MasVnrArea'].fillna(df['MasVnrArea'].mean())
for col in ['MasVnrType','FireplaceQu','GarageFinish','GarageQual',
'GarageCond','GarageType','BsmtExposure','BsmtFinType2',
'BsmtCond','BsmtFinType1','BsmtQual','Electrical']:
df[col] = df[col].fillna(df[col].mode()[0])

# EDA — Sale Price Distribution
sns.histplot(df['SalePrice'], kde=True)
plt.title('Sale Price Distribution'); plt.show()

# EDA — Correlation Heatmap
correlation = df.corr(numeric_only=True)
plt.figure(figsize=(12, 8))
sns.heatmap(correlation); plt.title('Correlation Heatmap'); plt.show()

# Encode & Split
df_encoded = pd.get_dummies(df, drop_first=True)
x = df_encoded.drop('SalePrice', axis=1)
y = df_encoded['SalePrice']
X_train, X_test, y_train, y_test = train_test_split(
x, y, test_size=0.2, random_state=42)

# Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print('MSE =', mse, ' | RMSE =', rmse)

# Ridge Regression
ridge = Ridge(alpha=0.1)
ridge.fit(X_train, y_train)
```

```
ridge_pred = ridge.predict(X_test)
ridge_mse = mean_squared_error(y_test, ridge_pred)
ridge_rmse = np.sqrt(ridge_mse)
print('Ridge MSE =', ridge_mse, ' | RMSE =', ridge_rmse)

# Lasso Regression
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
lasso_pred = lasso.predict(X_test)
lasso_mse = mean_squared_error(y_test, lasso_pred)
lasso_rmse = np.sqrt(lasso_mse)
print('Lasso MSE =', lasso_mse, ' | RMSE =', lasso_rmse)

# K-Means – Elbow Method
inertia = []
for k in range(1, 11):
km = KMeans(n_clusters=k, random_state=42)
km.fit(x)
inertia.append(km.inertia_)
plt.plot(range(1,11), inertia, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters'); plt.ylabel('Inertia'); plt.show()

# K-Means – Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(x)
df_encoded['Cluster'] = clusters
df_encoded.groupby('Cluster')['SalePrice'].mean()

# K-Means – Boxplot
sns.boxplot(x='Cluster', y='SalePrice', data=df_encoded)
plt.title('Sale Price by Cluster'); plt.show()
```